

C# Concepts Overview

In this session, we will cover several fundamental concepts in C#, including **methods**, **properties**, **constructors**, **object initializers**, **destructors**, and the **IDisposable** interface. Each section will provide detailed explanations, examples, and important points to help you understand these concepts thoroughly.

1. Methods

1.1 Method Overloading

- **Definition:** Method overloading allows multiple methods in the same class to have the same name but different parameter lists.
- **Purpose:** It provides flexibility in method calls based on different input parameters.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1public class MathOperations
2{
3    public int Add(int a, int b) => a + b;
4    public int Add(int a, int b, int c) => a + b + c;
5    public double Add(double a, double b) => a + b;
6}
```

1.2 Optional Parameters

- **Definition:** Optional parameters allow you to specify default values for parameters in a method.
- **Usage:** Must be defined after all required parameters.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1public void SayHello(string name, string greeting = "Hello", int times = 1)
2{
3    for (int i = 0; i < times; i++)
4    {
5        Console.WriteLine($"{greeting}, {name}!");
6    }
7}
```

1.3 Named Parameters

- **Definition:** Named parameters allow you to specify arguments by name, improving readability and allowing you to skip optional parameters.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1SayHello("Alice", times: 3); // Uses named argument for times
```

2. Properties

2.1 Get and Set Accessors

- **Definition:** Properties encapsulate data, allowing controlled access to class fields.
- **Get:** Retrieves the value.
- **Set:** Assigns a value.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1public class Person
2{
3    private string _name;
4    public string Name
5    {
6        get { return _name; }
7        set { _name = value; }
8    }
9}
```

2.2 Read-Only Properties

- **Definition:** Properties that only have a `get` accessor, preventing modification after initialization.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1public string Name { get; }
2public Person(string name) => Name = name; // Set during construction
```

2.3 Auto-Implemented Properties

- **Definition:** Simplifies property declarations without needing a backing field.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1public string Name { get; } // Auto-implemented read-only property
```

3. Constructors

3.1 What is a Constructor?

- **Definition:** A special method called when an instance of a class is created, used to initialize the object.

3.2 Types of Constructors

1. **Default Constructor:** No parameters.
2. **Parameterized Constructor:** Takes parameters to initialize fields.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1public class Person
2{
3    public string Name { get; set; }
4    public Person() => Name = "Unknown"; // Default
5    public Person(string name) => Name = name; // Parameterized
6}
```

3.3 Object Initializers

- **Definition:** Allows setting properties during object creation without calling a constructor explicitly.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1var person = new Person { Name = "Bob" };
```

4. Destructors

4.1 What is a Destructor?

- **Definition:** A special method called when an object is about to be destroyed, used for cleanup.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1~ResourceHolder() => Dispose(false); // Finalizer
```

5. IDisposable Interface

5.1 Purpose of IDisposable

- **Definition:** Provides a mechanism for releasing unmanaged resources explicitly.

5.2 Implementing IDisposable

- **Steps:**
 1. Implement IDisposable.
 2. Create a Dispose() method.
 3. Use a Dispose(bool disposing) method for cleanup logic.
 4. Call GC.SuppressFinalize(this).

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1public class FileHandler : IDisposable
2{
3    private FileStream _fileStream;
```

```
4     private bool _disposed = false;
5
6     public void Dispose()
7     {
8         Dispose(true);
9         GC.SuppressFinalize(this);
10    }
11
12    protected virtual void Dispose(bool disposing)
13    {
14        if (_disposed) return;
15        if (disposing) _fileStream?.Close();
16        _disposed = true;
17    }
18}
```

Conclusion

This overview covers essential C# concepts, including methods, properties, constructors, destructors, and the IDisposable interface. Understanding these concepts will enhance your ability to write clean, efficient, and maintainable code. If you have any specific questions or need further clarification on any topic, feel free to ask!